

Theory of Computation

Lecture 4: Nondeterminism and Nondeterministic Finite Automata

Sheikh Azizul Hakim

Lecturer, Department of CSE, BUET

Last compiled: June 28, 2026 7:38pm Z

Sometimes finding a solution is hard.
But verifying a guessed solution is easy.

Lecture 4A: The Philosophy of Nondeterminism

The Central Question

In deterministic computation, every step is forced.

(current state, input symbol) $\longrightarrow$ exactly one next state

Today we ask:

What if a machine could explore many possibilities?

The Central Question

In deterministic computation, every step is forced.

(current state, input symbol) $\longrightarrow$ exactly one next state

Today we ask:

What if a machine could explore many possibilities?

This idea is called **nondeterminism**.

Find versus Verify

Many computational tasks have two versions.

Problem	Find	Verify
Maze	Find a path	Check a given path
Sudoku	Solve puzzle	Check a completed board
Password	Discover password	Test a candidate password
SAT	Find assignment	Check a given assignment
Proof	Find proof	Check a written proof

Find versus Verify

Many computational tasks have two versions.

Problem	Find	Verify
Maze	Find a path	Check a given path
Sudoku	Solve puzzle	Check a completed board
Password	Discover password	Test a candidate password
SAT	Find assignment	Check a given assignment
Proof	Find proof	Check a written proof

The second task is often much easier.

Example: Maze

Task:

Find a path from the entrance to the exit.

This may require search.

Example: Maze

Task:

Find a path from the entrance to the exit.

This may require search.

But if someone gives us a candidate path, we can quickly check:

- Does it start at the entrance?
- Does every step follow a valid corridor?
- Does it end at the exit?

Example: Maze

Task:

Find a path from the entrance to the exit.

This may require search.

But if someone gives us a candidate path, we can quickly check:

- Does it start at the entrance?
- Does every step follow a valid corridor?
- Does it end at the exit?

Finding and verifying are different kinds of computation.

Example: Sudoku

Finding a solution may be difficult.

Verifying a completed board is straightforward:

- Check every row.
- Check every column.
- Check every block.

Example: Sudoku

Finding a solution may be difficult.

Verifying a completed board is straightforward:

- Check every row.
- Check every column.
- Check every block.

This distinction will reappear later in complexity theory as a central theme.

The Nondeterministic Dream

Imagine a machine that can say:

Let me guess the correct branch of the search.

If the guess can be verified, the machine accepts.

The Nondeterministic Dream

Imagine a machine that can say:

Let me guess the correct branch of the search.

If the guess can be verified, the machine accepts.

This is not how ordinary computers physically work.

But it is an extremely useful mathematical abstraction.

Three Ways to Think About Nondeterminism

1. **Magical guessing**

The machine guesses the correct choice.

2. **Parallel universes**

All choices are explored simultaneously.

3. **Computation tree**

The computation branches into many paths.

Three Ways to Think About Nondeterminism

1. **Magical guessing**

The machine guesses the correct choice.

2. **Parallel universes**

All choices are explored simultaneously.

3. **Computation tree**

The computation branches into many paths.

For finite automata, these viewpoints describe the same idea.

Does Nondeterminism Exist Physically?

Probably not as a literal machine that always guesses correctly.

But theoretical computer science often studies idealized models.

Examples:

- infinite tapes,
- real numbers,
- perfect randomness,
- nondeterministic choices.

Does Nondeterminism Exist Physically?

Probably not as a literal machine that always guesses correctly.

But theoretical computer science often studies idealized models.

Examples:

- infinite tapes,
- real numbers,
- perfect randomness,
- nondeterministic choices.

Idealized models help us understand the structure and limits of computation.

Why Study It Then?

Nondeterminism models the shape of search.

It appears behind many real computational tools:

- backtracking algorithms,
- SAT solvers,
- theorem provers,
- symbolic execution,
- program analysis,
- AI planning,
- regex engines.

**Nondeterminism is not merely an automata trick.
It is one of the central ideas of theoretical computer science.**

Lecture 4B: From DFA to NFA

Recall: DFA

A deterministic finite automaton has

$$M = (Q, \Sigma, \delta, q_0, F).$$

The transition function is

$$\delta : Q \times \Sigma \rightarrow Q.$$

For each state and input symbol, there is exactly one next state.

What Changes in an NFA?

In a nondeterministic finite automaton, one situation may have many possible next states.

Instead of

$$(q, a) \mapsto q'$$

we allow

$$(q, a) \mapsto \{q_1, q_2, \dots, q_k\}.$$

The machine may branch.

Let

$$\text{CONTAINS}_{101} = \{w \in \{0,1\}^* : w \text{ contains } 101 \text{ as a substring}\}.$$

Examples in the language:

101, 0101, 1010, 1110100.

Examples not in the language:

ε , 0, 11, 1000, 0100.

To recognize whether the input contains 101, a DFA keeps track of partial progress.

It remembers whether it has recently seen:

nothing useful, 1, 10, 101.

This is perfectly possible.

To recognize whether the input contains 101, a DFA keeps track of partial progress.

It remembers whether it has recently seen:

nothing useful, 1, 10, 101.

This is perfectly possible.

But there is another way to think.

Whenever the machine sees a 1, it may guess:

This 1 is the beginning of 101.

If the guess is correct, one branch reaches an accepting state.

If the guess is wrong, that branch dies.

Whenever the machine sees a 1, it may guess:

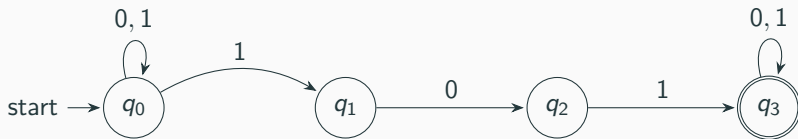
This 1 is the beginning of 101.

If the guess is correct, one branch reaches an accepting state.

If the guess is wrong, that branch dies.

The input is accepted if at least one guess succeeds.

NFA for Strings Containing 101



The branch $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3$ verifies one guessed occurrence of 101.

Three Outcomes of a Branch

In an NFA, a single input may create many computation branches.

A branch can end in three different ways:

Outcome	Meaning
Accepted	input is fully read and the branch ends in an accepting state
Rejected	input is fully read but the branch ends in a non-accepting state
Dead	the branch gets stuck before the input is fully read

Three Outcomes of a Branch

In an NFA, a single input may create many computation branches.

A branch can end in three different ways:

Outcome	Meaning
Accepted	input is fully read and the branch ends in an accepting state
Rejected	input is fully read but the branch ends in a non-accepting state
Dead	the branch gets stuck before the input is fully read

The NFA accepts the input if **at least one** branch is accepted.

Why Is This Nondeterministic?

At state q_0 , on input 1, the machine has two choices:

$$\delta(q_0, 1) = \{q_0, q_1\}.$$

Interpretation:

- q_0 : ignore this 1;
- q_1 : guess this 1 starts the pattern 101.

Why Is This Nondeterministic?

At state q_0 , on input 1, the machine has two choices:

$$\delta(q_0, 1) = \{q_0, q_1\}.$$

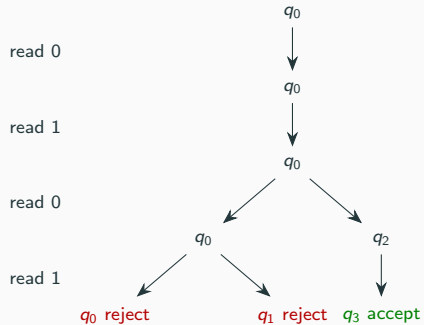
Interpretation:

- q_0 : ignore this 1;
- q_1 : guess this 1 starts the pattern 101.

Both possibilities are explored as separate branches.

Computation Tree for 0101

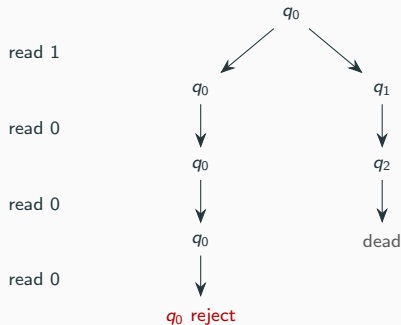
Input: 0101



One leaf reaches $q_3 \in F$, so 0101 is accepted.

Computation Tree for 1000

Input: 1000



The guessed 101-branch dies at q_2 on input 0. The surviving branch ends at non-accepting q_0 . So 1000 is rejected.

Important rule:

An NFA accepts if at least one computation branch accepts.

- Accepted branch: input consumed, final state is in F .
- Rejected branch: input consumed, final state is not in F .
- Dead branch: no valid transition before the input is consumed.

Important rule:

An NFA accepts if at least one computation branch accepts.

- Accepted branch: input consumed, final state is in F .
- Rejected branch: input consumed, final state is not in F .
- Dead branch: no valid transition before the input is consumed.

Many bad branches do not matter if one good branch accepts.

Proving Acceptance versus Rejection

To show that an NFA **accepts** a string, it is enough to show one accepting branch.

Claim	What must be shown?
w is accepted	exhibit one valid accepting branch
w is rejected	rule out every possible accepting branch

Proving Acceptance versus Rejection

To show that an NFA **accepts** a string, it is enough to show one accepting branch.

Claim	What must be shown?
w is accepted	exhibit one valid accepting branch
w is rejected	rule out every possible accepting branch

This is the “magical guessing” viewpoint:

If an accepting branch exists, the NFA may guess that branch.

Proving Acceptance versus Rejection

To show that an NFA **accepts** a string, it is enough to show one accepting branch.

Claim	What must be shown?
w is accepted	exhibit one valid accepting branch
w is rejected	rule out every possible accepting branch

This is the “magical guessing” viewpoint:

If an accepting branch exists, the NFA may guess that branch.

But for rejection, there is no shortcut:

Every branch must be checked, and none may accept.

A DFA has one timeline.

An NFA has many possible timelines.

Formal Definition

A nondeterministic finite automaton is a 5-tuple

$$M = (Q, \Sigma, \delta, q_0, F),$$

where:

- Q is a finite set of states,
- Σ is the input alphabet,
- $q_0 \in Q$ is the start state,
- $F \subseteq Q$ is the set of accepting states,
- $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ is the transition function.

Only δ differs from the DFA case.

DFA versus NFA Transition Functions

For a DFA:

$$\delta : Q \times \Sigma \rightarrow Q.$$

For an NFA:

$$\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q).$$

Here $\mathcal{P}(Q)$ means the set of all subsets of Q .

Why a Set of States?

Suppose

$$Q = \{q_0, q_1, q_2\}.$$

Then

$$\mathcal{P}(Q) = \{\emptyset, \{q_0\}, \{q_1\}, \{q_2\}, \{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}.$$

An NFA transition may return any one of these subsets.

Interpreting the Output of δ

$$\delta(q, a) = \emptyset$$

means no branch survives.

$$\delta(q, a) = \{r\}$$

means exactly one next possibility.

$$\delta(q, a) = \{r, s, t\}$$

means the computation branches into three possibilities.

Formal Specification of the 101 NFA

$$Q = \{q_0, q_1, q_2, q_3\}, \quad \Sigma = \{0, 1\}, \quad F = \{q_3\}.$$

q_0 is the start state.

The transition function is:

δ	0	1
q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_2\}$	$\emptyset$
q_2	$\emptyset$	$\{q_3\}$
q_3	$\{q_3\}$	$\{q_3\}$

Notice the Difference

In a DFA table, each entry is a state.

$q_0, \quad q_1, \quad q_2$

In an NFA table, each entry is a set of states.

$\emptyset, \quad \{q_0\}, \quad \{q_0, q_1\}$

The table is just a precise way to define the transition function.

Every DFA Is an NFA

A DFA transition

$$\delta(q, a) = r$$

can be viewed as the NFA transition

$$\delta(q, a) = \{r\}.$$

So every DFA is automatically an NFA.

Every DFA Is an NFA

A DFA transition

$$\delta(q, a) = r$$

can be viewed as the NFA transition

$$\delta(q, a) = \{r\}.$$

So every DFA is automatically an NFA.

But is every NFA equivalent to some DFA?

That is the big question.

Lecture 4C: More Examples

Example 1: Contains *fed* or *dff*

Language over $\Sigma = \{d, e, f\}$:

$$L = \{w \in \{d, e, f\}^* : w \text{ contains } \textit{fed} \text{ or } \textit{dff} \text{ as a substring}\}.$$

Members: *fed*, *dff*, *efed*, *dffd*, *ffdff*

Non-members: ε , *d*, *ff*, *def*, *fef*, *edf*

Example 1: Contains *fed* or *dff*

Language over $\Sigma = \{d, e, f\}$:

$$L = \{w \in \{d, e, f\}^* : w \text{ contains } \textit{fed} \text{ or } \textit{dff} \text{ as a substring}\}.$$

Members: *fed*, *dff*, *efed*, *dffed*, *ffdff*

Non-members: ε , *d*, *ff*, *def*, *fef*, *edf*

The NFA guesses where one of the two target substrings begins.

NFA Idea: Two Possible Patterns

We want to detect either

fed or *dff*.

The machine scans the input in state q_0 .

- When it sees f , it may guess that *fed* is beginning.
- When it sees d , it may guess that *dff* is beginning.

NFA Idea: Two Possible Patterns

We want to detect either

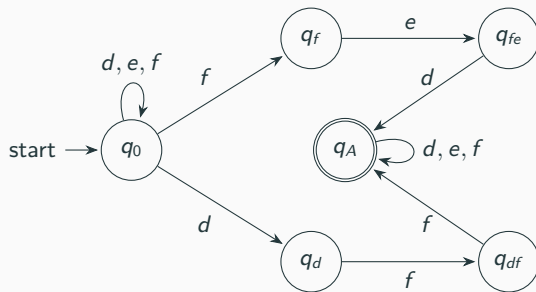
fed or *dff*.

The machine scans the input in state q_0 .

- When it sees f , it may guess that *fed* is beginning.
- When it sees d , it may guess that *dff* is beginning.

If either guessed pattern is completed, the input is accepted.

NFA for Contains fed or dff



Missing transitions mean that the corresponding branch dies.

Example 2: Guessing a Split Point

Language over $\Sigma = \{a, b\}$:

$$L = \{w_1 w_2 : |w_1| \text{ is a multiple of 3 and } w_2 \text{ has an even number of } b\text{'s}\}.$$

$w_1 w_2$ means the concatenation of w_1 and w_2

Members: ε , a , aa , bb , aaa , aab , $abba$

Non-members: b , ab , ba , $aaab$, $abbb$

Example 2: Guessing a Split Point

Language over $\Sigma = \{a, b\}$:

$$L = \{w_1 w_2 : |w_1| \text{ is a multiple of 3 and } w_2 \text{ has an even number of } b\text{'s}\}.$$

$w_1 w_2$ means the concatenation of w_1 and w_2

Members: ε , a , aa , bb , aaa , aab , $abba$

Non-members: b , ab , ba , $aaab$, $abbb$

The NFA guesses the split point, but without using an ε -transition.

The Intuition

The top part tracks the length of the prefix modulo 3:

$$r_0, \quad r_1, \quad r_2.$$

A split is allowed only when the prefix length is a multiple of 3, that is, at r_0 .

The top part tracks the length of the prefix modulo 3:

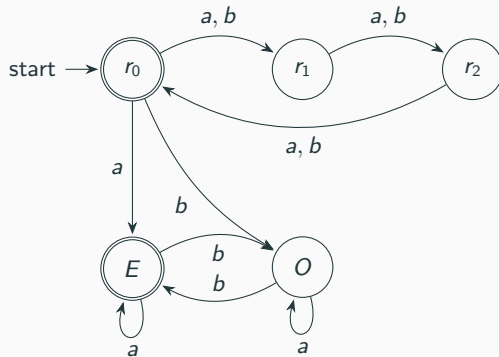
$$r_0, \quad r_1, \quad r_2.$$

A split is allowed only when the prefix length is a multiple of 3, that is, at r_0 .

From r_0 , on the next symbol, the NFA has two choices:

- keep reading w_1 ;
- treat this symbol as the first symbol of w_2 .

NFA for Guessing the Split Point



State r_0 is accepting because the NFA may choose $w_2 = \varepsilon$.

Reading the Split-Point NFA

The states r_0, r_1, r_2 track

$$|w_1| \bmod 3.$$

The states E, O track whether the guessed suffix w_2 has an even or odd number of b 's.

Reading the Split-Point NFA

The states r_0, r_1, r_2 track

$$|w_1| \bmod 3.$$

The states E, O track whether the guessed suffix w_2 has an even or odd number of b 's.

At r_0 , the NFA may begin reading w_2 :

$$r_0 \xrightarrow{a} E, \quad r_0 \xrightarrow{b} O.$$

No ε -transition is used.

Example 3: Missing at Least One Symbol

Language over $\Sigma = \{a, b, c\}$:

$$L = \{w \in \{a, b, c\}^* : w \text{ is missing at least one of } a, b, c\}.$$

Members: ε , a , ab , $bbbb$, $acac$, $ccaa$

Non-members: abc , acb , $baac$, $cbabc$

Example 3: Missing at Least One Symbol

Language over $\Sigma = \{a, b, c\}$:

$$L = \{w \in \{a, b, c\}^* : w \text{ is missing at least one of } a, b, c\}.$$

Members: ε , a , ab , $bbbb$, $acac$, $ccaa$

Non-members: abc , acb , $baac$, $cbabc$

The NFA guesses which symbol will be missing.

NFA Idea: Guess the Missing Symbol

The string is in the language if

it has no a or it has no b or it has no c .

The NFA branches into three possible checks:

$q_{\neg a}$, $q_{\neg b}$, $q_{\neg c}$.

NFA Idea: Guess the Missing Symbol

The string is in the language if

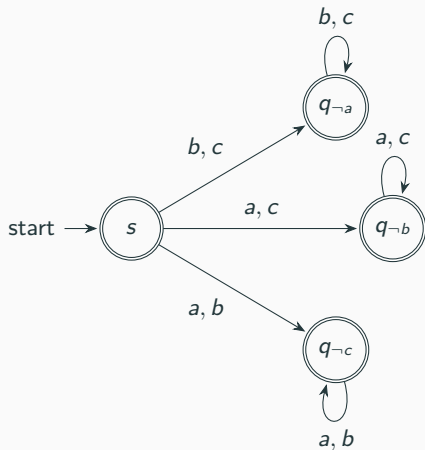
it has no a or it has no b or it has no c .

The NFA branches into three possible checks:

$q_{\neg a}$, $q_{\neg b}$, $q_{\neg c}$.

Without ε -transitions, the branching happens while reading the first input symbol.

NFA for Missing at Least One Symbol



State s accepts ϵ . A branch dies if it reads the symbol it guessed would be missing.

Transition Table for the Missing-Symbol NFA

All four states shown here are accepting.

State	on a	on b	on c
s	$\{q_{\neg b}, q_{\neg c}\}$	$\{q_{\neg a}, q_{\neg c}\}$	$\{q_{\neg a}, q_{\neg b}\}$
$q_{\neg a}$	$\emptyset$	$\{q_{\neg a}\}$	$\{q_{\neg a}\}$
$q_{\neg b}$	$\{q_{\neg b}\}$	$\emptyset$	$\{q_{\neg b}\}$
$q_{\neg c}$	$\{q_{\neg c}\}$	$\{q_{\neg c}\}$	$\emptyset$

For example, on input abc , every branch eventually dies.

What These Examples Show

- For substring search, an NFA guesses where the pattern begins.
- For split languages, an NFA guesses where one part ends and the next begins.
- For missing-symbol languages, an NFA guesses which condition will remain true.

What These Examples Show

- For substring search, an NFA guesses where the pattern begins.
- For split languages, an NFA guesses where one part ends and the next begins.
- For missing-symbol languages, an NFA guesses which condition will remain true.

The common theme:

An NFA is good at expressing “there exists a good choice.”

Lecture 4D: Epsilon Transitions

Epsilon-NFA

An **epsilon-NFA** is an NFA that may move without reading input.

Formally, it is a 5-tuple

$$M = (Q, \Sigma, \delta, q_0, F),$$

where

$$\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q), \quad \Sigma_\epsilon = \Sigma \cup \{\epsilon\}.$$

So a transition may be labeled by an ordinary input symbol, or by ϵ .

$$q \xrightarrow{\epsilon} r$$

means: move from q to r without consuming any input.

How Does an Epsilon-NFA Accept?

Let $w \in \Sigma^*$. An epsilon-NFA accepts w if there exists a path

$$q_0 = p_0 \rightarrow p_1 \rightarrow \cdots \rightarrow p_m$$

such that:

1. each transition label is either a symbol from Σ or ε ;
2. after deleting all ε 's from the labels, the remaining string is exactly w ;
3. the final state is accepting: $p_m \in F$.

How Does an Epsilon-NFA Accept?

Let $w \in \Sigma^*$. An epsilon-NFA accepts w if there exists a path

$$q_0 = p_0 \rightarrow p_1 \rightarrow \cdots \rightarrow p_m$$

such that:

1. each transition label is either a symbol from Σ or ε ;
2. after deleting all ε 's from the labels, the remaining string is exactly w ;
3. the final state is accepting: $p_m \in F$.

In short:

The machine may take free ε -moves before, between, or after reading symbols.

Why ε -Transitions Are Useful

They make automata modular.

Suppose we want to recognize

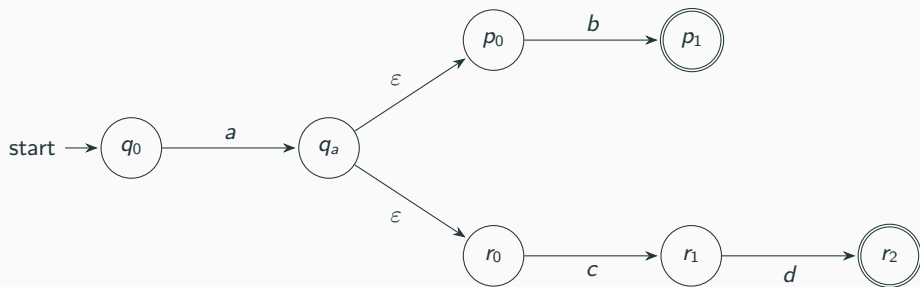
$$L = \{ab, acd\}.$$

Both strings begin with a . After reading a , the machine must choose:

finish with b or finish with cd .

With ε -transitions, we can first read the common prefix a , then freely jump into the appropriate suffix module.

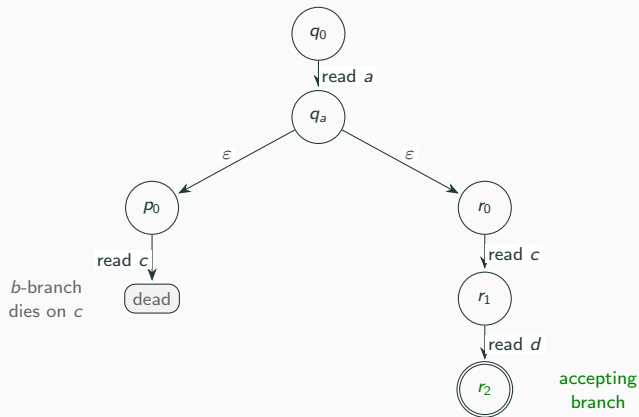
Epsilon-NFA for $\{ab, acd\}$



The ε -moves occur after the input symbol a has already been consumed.

Computation Tree on Input acd

Input: acd



The b -suffix branch dies on c , but the cd -suffix branch accepts.

One Accepting Computation Path

For input acd , one accepting path is:

$$q_0 \xrightarrow{a} q_a \xrightarrow{\varepsilon} r_0 \xrightarrow{c} r_1 \xrightarrow{d} r_2.$$

After deleting the ε -label, the consumed input is

acd .

Since $r_2 \in F$, this path accepts. Therefore, the epsilon-NFA accepts acd .

Epsilon-Closure

To remove ε -transitions, we need one definition.

The **epsilon-closure** of a state q , denoted $E(q)$, is:

$$E(q) = \{r \in Q : r \text{ is reachable from } q \text{ using zero or more } \varepsilon\text{-transitions}\}.$$

We include q itself: $q \in E(q)$.

For a set $S \subseteq Q$, define

$$E(S) = \bigcup_{q \in S} E(q).$$

Removing Epsilon Transitions

Every epsilon-NFA can be converted into an ordinary NFA.

Let $M = (Q, \Sigma, \delta, q_0, F)$ be an epsilon-NFA.

We build $M' = (Q, \Sigma, \delta', q_0, F')$ with no ε -transitions.

For each $q \in Q$ and $a \in \Sigma$, define

$$\delta'(q, a) = E \left(\bigcup_{r \in E(q)} \delta(r, a) \right).$$

This says: take free moves, read a , then take free moves again.

Accepting States After Removing ε

We also update the accepting states.

Define

$$F' = \{q \in Q : E(q) \cap F \neq \emptyset\}.$$

Why?

Accepting States After Removing ε

We also update the accepting states.

Define

$$F' = \{q \in Q : E(q) \cap F \neq \emptyset\}.$$

Why?

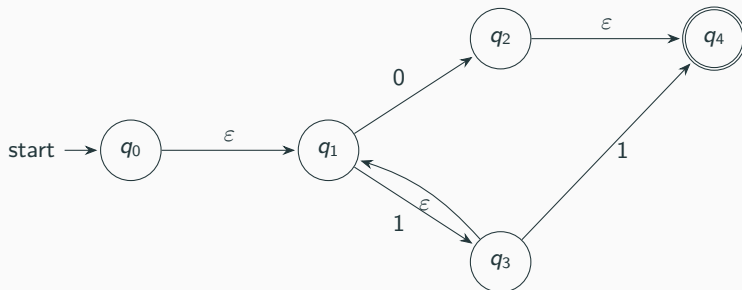
If a state q can reach an accepting state using only ε -moves, then q should be accepting in the new NFA.

This handles the case where the epsilon-NFA accepts after the input has already been consumed.

Example: An Epsilon-NFA over $\{0, 1\}$

Let

$$Q = \{q_0, q_1, q_2, q_3, q_4\}, \quad \Sigma = \{0, 1\}, \quad F = \{q_4\}.$$



All missing transitions go to $\emptyset$.

Step 1: Compute Epsilon-Closures

For the epsilon-NFA above:

State	Epsilon-closure
q_0	$E(q_0) = \{q_0, q_1\}$
q_1	$E(q_1) = \{q_1\}$
q_2	$E(q_2) = \{q_2, q_4\}$
q_3	$E(q_3) = \{q_3, q_1\}$
q_4	$E(q_4) = \{q_4\}$

Notice:

$$q_2 \in F'$$

because q_2 can reach the accepting state q_4 using only an ϵ -move.

Step 2: New Accepting States

Original accepting states: $F = \{q_4\}$.

After removing ε -transitions:

$$F' = \{q \in Q : E(q) \cap F \neq \emptyset\}.$$

From the closure table:

$$E(q_2) = \{q_2, q_4\}, \quad E(q_4) = \{q_4\}.$$

Therefore,

$$F' = \{q_2, q_4\}.$$

Step 3: New Transition Table

The converted NFA has no ε -transitions.

$$\delta'(q, a) = E \left(\bigcup_{r \in E(q)} \delta(r, a) \right).$$

State	on 0	on 1
q_0	$\{q_2, q_4\}$	$\{q_3, q_1\}$
q_1	$\{q_2, q_4\}$	$\{q_3, q_1\}$
q_2	$\emptyset$	$\emptyset$
q_3	$\{q_2, q_4\}$	$\{q_4, q_3, q_1\}$
q_4	$\emptyset$	$\emptyset$

Accepting states in the new NFA: $\{q_2, q_4\}$.

One Entry Explained

Question: Why is $\delta'(q_0, 0) = \{q_2, q_4\}$?

Start with the epsilon-closure. $E(q_0) = \{q_0, q_1\}$.

Then read 0:

$$\delta(q_0, 0) \cup \delta(q_1, 0) = \emptyset \cup \{q_2\} = \{q_2\}.$$

Then take epsilon-closure again:

$$E(\{q_2\}) = \{q_2, q_4\}.$$

So

$$\delta'(q_0, 0) = \{q_2, q_4\}.$$

What This Means

Epsilon transitions do not add new language-recognition power.

They only make constructions cleaner.

epsilon-NFA $\iff$ NFA without epsilon transitions

What This Means

Epsilon transitions do not add new language-recognition power.

They only make constructions cleaner.

epsilon-NFA $\iff$ NFA without epsilon transitions

So we are free to use ε -transitions when they make the design simpler.

A Note on Terminology

Different books use slightly different conventions.

In Sipser's textbook, an **NFA** is allowed to have ε -transitions.

From now on, we will follow that convention:

When we say NFA, we allow ε -transitions unless stated otherwise.

If we need to forbid them, we will explicitly say “NFA without ε -transitions.”

Lecture 4E: Equivalence of DFA and NFA

Definition: Two machines are equivalent if and only if they recognize the same language.

We already know that every DFA is automatically an NFA.

Definition: Two machines are equivalent if and only if they recognize the same language.

We already know that every DFA is automatically an NFA.

The surprising part: Every NFA has an equivalent DFA.

This means that adding nondeterminism does **not** increase the theoretical power of finite automata. They still recognize exactly the same set of languages, namely **the regular languages**.

The Subset Construction (Idea)

How can a deterministic machine simulate a nondeterministic one?

- An NFA can be in multiple states at once across different branches of computation.
- A DFA can only be in exactly one state.
- **Solution:** Make the DFA's states represent *sets* of NFA states!

The Subset Construction (Idea)

How can a deterministic machine simulate a nondeterministic one?

- An NFA can be in multiple states at once across different branches of computation.
- A DFA can only be in exactly one state.
- **Solution:** Make the DFA's states represent *sets* of NFA states!

If the NFA has Q as its set of states, the equivalent DFA will use the powerset $\mathcal{P}(Q)$ as its states.

If $|Q| = n$, the DFA will have up to 2^n states.

Note: We already showed that any NFA with ϵ -transitions can be converted to one without them. Therefore, for this proof, we assume our starting NFA has no ϵ -transitions.

Formal Construction

Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA without ϵ -transitions.

We construct an equivalent DFA $M = (Q', \Sigma, \delta', q'_0, F')$ as follows:

1. **States:** $Q' = \mathcal{P}(Q)$.
2. **Start state:** $q'_0 = \{q_0\}$ (The set containing only the NFA start state).
3. **Accepting states:**

$$F' = \{R \in Q' \mid R \cap F \neq \emptyset\}$$

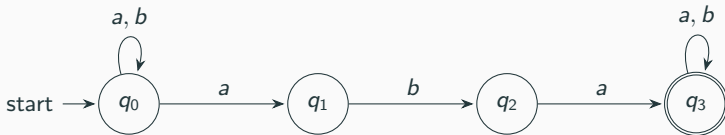
(Any subset containing at least one NFA accepting state).

4. **Transition function:** For any $R \in Q'$ and $a \in \Sigma$:

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$$

Worked Example

Let's convert an NFA that recognizes strings containing the substring *aba*.

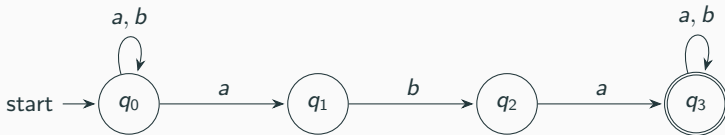


$$Q = \{q_0, q_1, q_2, q_3\}.$$

The theoretical equivalent DFA has states in $\mathcal{P}(Q)$, meaning $2^4 = 16$ possible states.

Worked Example

Let's convert an NFA that recognizes strings containing the substring *aba*.



$$Q = \{q_0, q_1, q_2, q_3\}.$$

The theoretical equivalent DFA has states in $\mathcal{P}(Q)$, meaning $2^4 = 16$ possible states.

Good news: We only need to construct the states that are actually *reachable* from the start state.

Step-by-Step Conversion: Setup and Initial States

1. Start State:

The DFA start state is simply the set containing the NFA start state: $A = \{q_0\}$.

Step-by-Step Conversion: Setup and Initial States

1. Start State:

The DFA start state is simply the set containing the NFA start state: $A = \{q_0\}$.

2. Transitions from $A = \{q_0\}$:

- On a : $\delta(q_0, a) = \{q_0, q_1\}$. $\rightarrow$ New state $B = \{q_0, q_1\}$.
- On b : $\delta(q_0, b) = \{q_0\}$. $\rightarrow$ Back to A .

Step-by-Step Conversion: Setup and Initial States

1. Start State:

The DFA start state is simply the set containing the NFA start state: $A = \{q_0\}$.

2. Transitions from $A = \{q_0\}$:

- On a : $\delta(q_0, a) = \{q_0, q_1\}$. $\rightarrow$ New state $B = \{q_0, q_1\}$.
- On b : $\delta(q_0, b) = \{q_0\}$. $\rightarrow$ Back to A .

3. Transitions from $B = \{q_0, q_1\}$:

- On a : $\delta(q_0, a) \cup \delta(q_1, a) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$. $\rightarrow$ Back to B .
- On b : $\delta(q_0, b) \cup \delta(q_1, b) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$. $\rightarrow$ New state $C = \{q_0, q_2\}$.

Step-by-Step Conversion: The Domino Effect

4. Transitions from $C = \{q_0, q_2\}$:

- On a : $\delta(q_0, a) \cup \delta(q_2, a) = \{q_0, q_1\} \cup \{q_3\} = \{q_0, q_1, q_3\}$. $\rightarrow$ New state D .
- On b : $\delta(q_0, b) \cup \delta(q_2, b) = \{q_0\} \cup \emptyset = \{q_0\}$. $\rightarrow$ Back to A .

Step-by-Step Conversion: The Domino Effect

4. Transitions from $C = \{q_0, q_2\}$:

- On a : $\delta(q_0, a) \cup \delta(q_2, a) = \{q_0, q_1\} \cup \{q_3\} = \{q_0, q_1, q_3\}$. $\rightarrow$ New state D .
- On b : $\delta(q_0, b) \cup \delta(q_2, b) = \{q_0\} \cup \emptyset = \{q_0\}$. $\rightarrow$ Back to A .

5. Transitions from $D = \{q_0, q_1, q_3\}$:

(Note: Since $q_3 \in D$, D is an accepting state!)

- On a : $\{q_0, q_1\} \cup \emptyset \cup \{q_3\} = \{q_0, q_1, q_3\}$. $\rightarrow$ Back to D .
- On b : $\{q_0\} \cup \{q_2\} \cup \{q_3\} = \{q_0, q_2, q_3\}$. $\rightarrow$ New state E .

Step-by-Step Conversion: The Domino Effect

4. Transitions from $C = \{q_0, q_2\}$:

- On a : $\delta(q_0, a) \cup \delta(q_2, a) = \{q_0, q_1\} \cup \{q_3\} = \{q_0, q_1, q_3\}$. $\rightarrow$ New state D .
- On b : $\delta(q_0, b) \cup \delta(q_2, b) = \{q_0\} \cup \emptyset = \{q_0\}$. $\rightarrow$ Back to A .

5. Transitions from $D = \{q_0, q_1, q_3\}$:

(Note: Since $q_3 \in D$, D is an accepting state!)

- On a : $\{q_0, q_1\} \cup \emptyset \cup \{q_3\} = \{q_0, q_1, q_3\}$. $\rightarrow$ Back to D .
- On b : $\{q_0\} \cup \{q_2\} \cup \{q_3\} = \{q_0, q_2, q_3\}$. $\rightarrow$ New state E .

Wait, another state? Let's keep going...

Step-by-Step Conversion: Reaching Closure

6. Transitions from $E = \{q_0, q_2, q_3\}$: (Accepting)

- On a : $\{q_0, q_1\} \cup \{q_3\} \cup \{q_3\} = \{q_0, q_1, q_3\}$. $\rightarrow$ Goes to D .
- On b : $\{q_0\} \cup \emptyset \cup \{q_3\} = \{q_0, q_3\}$. $\rightarrow$ New state F .

Step-by-Step Conversion: Reaching Closure

6. Transitions from $E = \{q_0, q_2, q_3\}$: (Accepting)

- On a : $\{q_0, q_1\} \cup \{q_3\} \cup \{q_3\} = \{q_0, q_1, q_3\}$. $\rightarrow$ Goes to D .
- On b : $\{q_0\} \cup \emptyset \cup \{q_3\} = \{q_0, q_3\}$. $\rightarrow$ New state F .

7. Transitions from $F = \{q_0, q_3\}$: (Accepting)

- On a : $\{q_0, q_1\} \cup \{q_3\} = \{q_0, q_1, q_3\}$. $\rightarrow$ Goes to D .
- On b : $\{q_0\} \cup \{q_3\} = \{q_0, q_3\}$. $\rightarrow$ Back to F .

Step-by-Step Conversion: Reaching Closure

6. Transitions from $E = \{q_0, q_2, q_3\}$: (Accepting)

- On a : $\{q_0, q_1\} \cup \{q_3\} \cup \{q_3\} = \{q_0, q_1, q_3\}$. $\rightarrow$ Goes to D .
- On b : $\{q_0\} \cup \emptyset \cup \{q_3\} = \{q_0, q_3\}$. $\rightarrow$ New state F .

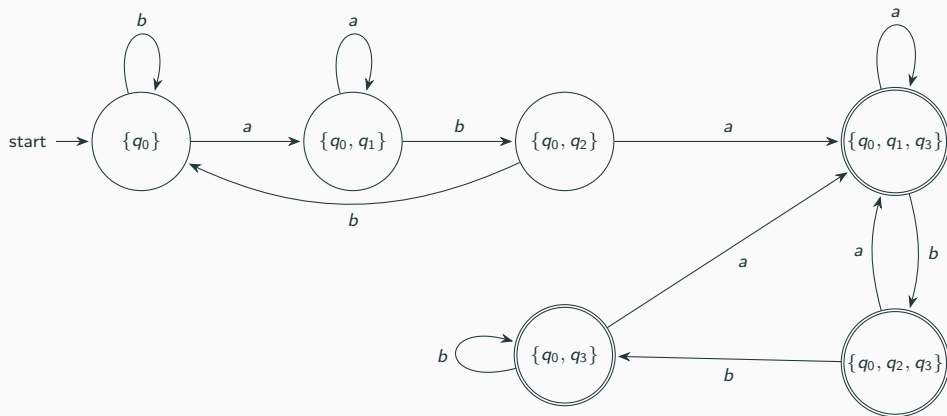
7. Transitions from $F = \{q_0, q_3\}$: (Accepting)

- On a : $\{q_0, q_1\} \cup \{q_3\} = \{q_0, q_1, q_3\}$. $\rightarrow$ Goes to D .
- On b : $\{q_0\} \cup \{q_3\} = \{q_0, q_3\}$. $\rightarrow$ Back to F .

Done! No new states were generated. Out of 16 possible subsets, only 6 are reachable from the start state.

The Equivalent DFA

Here is the deterministic machine we just built. It elegantly tracks the longest matching prefix of the target substring *aba*.



There can be smaller equivalent DFAs than the ones we can find using this explicit construction.

However, we need a generalized procedure to show that every language recognized by some NFA can also be recognized by some DFA.